



RTDIP x



Build and Deploy Documentation



RTDIP

x



Overview

This document describes the build and deployment process for the RTDIP forecasting pipeline. It provides step-by-step instructions to set up the development environment, build and run the project using VS Code Dev Containers, and configure the required dependencies and credentials. The goal is to ensure a reproducible and consistent setup for developers working on data ingestion, preprocessing, model training, and visualization components of the RTDIP forecasting system.

Prerequisites

Dev container built and running (VS Code Dev Containers)

Conda environment activated: `conda activate rtdip-sdk`

Azure credentials configured for data ingestion from Azure Blob Storage

Build Process

1. Clone Repository:

```
git clone https://github.com/amosproj/amos2025ws03-rtdip-timeseries-forecasting.git
```

```
cd amos2025ws03-rtdip-timeseries-forecasting
```

2. Build Dev Container:

VS Code detects `.devcontainer/devcontainer.json`

Click "Reopen in Container" when prompted

Manual rebuild: `Ctrl+Shift+P` → "Dev Containers: Rebuild Container"

3. Activate Environment:

```
conda activate rtdip-sdk
```

Testing

- Run all tests

```
pytest tests/ -v
```

- Run with coverage

```
pytest --cov=. --cov-report=xml:cov.xml tests/
```



RTDIP X



- [Run specific test modules](#)

```
pytest tests/sdk/python/rtdip_sdk/pipelines/forecasting/ -v
```

SDK Integration

```
from rtdip_sdk.pipelines.sources import PythonAzureBlobSource
from rtdip_sdk.pipelines.forecasting.spark import LSTMTimeSeries
from rtdip_sdk.pipelines.visualization.plotly.forecasting import
ForecastPlotInteractive
```

- Data Ingestion

```
source = PythonAzureBlobSource(
    account_url="https://<account>.blob.core.windows.net",
    container_name="<container>",
    credential="<sas_token>",
    file_pattern="*.parquet",
    combine_blobs=True,
    eager=True
)
df = source.read_batch()
```

- LSTM Model

```
lstm = LSTMTimeSeries(
    prediction_length=24,
    lookback_window=72,
    lstm_units=48,
    num_lstm_layers=2,
    embedding_dim=8
)
lstm.train(train_spark_df)
predictions = lstm.predict(test_spark_df)
metrics = lstm.evaluate(test_spark_df)
```

- Visualization

```
plot = ForecastPlotInteractive(
    historical_data=historical_df,
    forecast_data=forecast_df,
    sensor_id="TAG_001"
)
plot.plot()
plot.save("forecast.html")
```